Day 2

Python Web APIs

Service Desk Lite



© 2026 Teerasej Jiraphatchandej. All rights reserved.

A Web API gives apps a shared, controlled way to access services



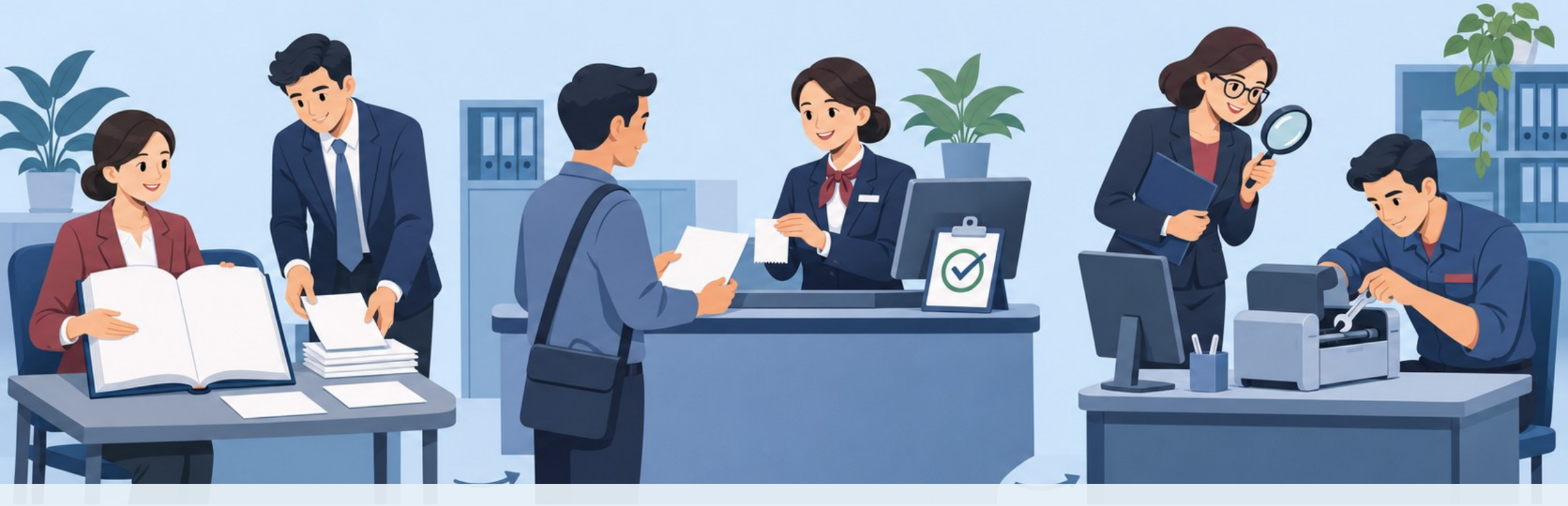
© 2026 Teerasej Jiraphatchandej. All rights reserved.

A Web API is a service counter for software



© 2026 Teerasej Jiraphatchandej. All rights reserved.

Build the API, test it, and keep it reliable



© 2026 Teerasej Jiraphatchandej. All rights reserved.

Exercise 1

Python bridge and a first API



© 2026 Teerasej Jiraphatchandej. All rights reserved.

Client, API and database

Each layer has a different job

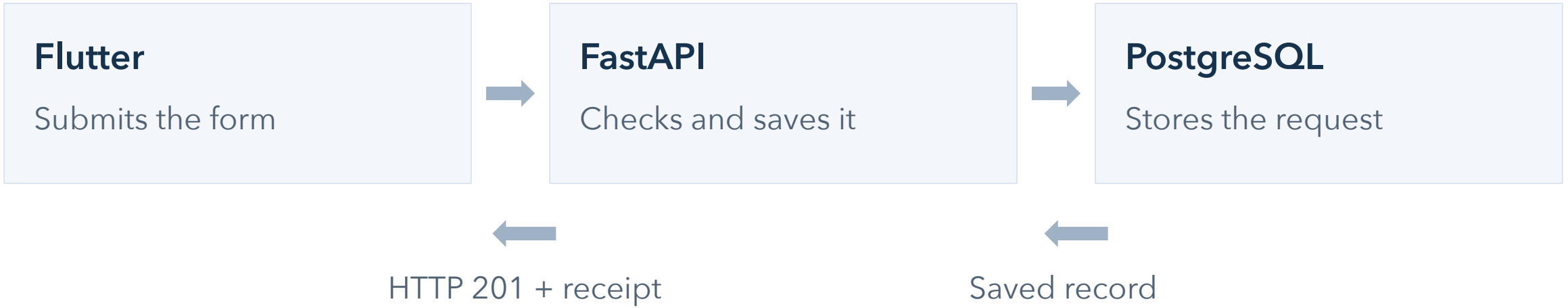


Flutter calls the API; the API talks to PostgreSQL

© 2026 Teerasej Jiraphatchandej. All rights reserved.

A projector request makes a round trip

Follow one request through the finished app.



The receipt gives Flutter the ID of the request that was saved.

© 2026 Teerasej Jiraphatchandej. All rights reserved.

Who does what at the service counter?

The office counter gives each layer a clear job.

At the office	In Service Desk Lite	Job
Request form	Flutter client	Collect details and show the receipt
Service counter	FastAPI	Check the request and access rules
Records cabinet	PostgreSQL	Keep the saved record

FastAPI holds the database connection. Flutter talks to FastAPI.

© 2026 Teerasej Jiraphatchandej. All rights reserved.

HTTP request and response

A request asks the service to do something

A response returns a status and data

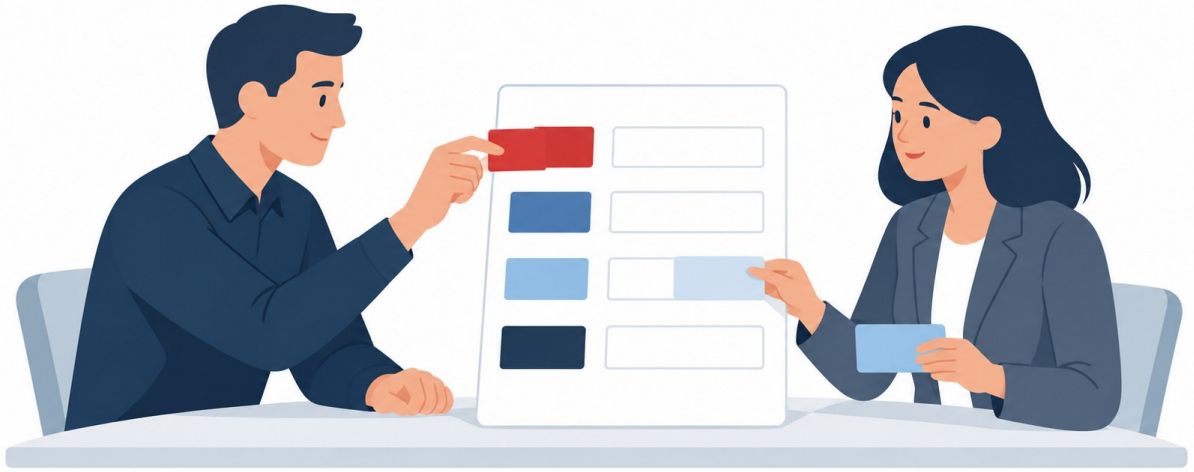


© 2026 Teerasej Jiraphatchandej. All rights reserved.

JSON

JSON organizes data as names and values

Sender and receiver need the same structure



© 2026 Teerasej Jiraphatchandej. All rights reserved.

From form fields to JSON

The field name becomes the key. The form entry becomes its value.

Projector request form

Title	Fix projector
Description	Meeting room projector is blank.
Category	Equipment

JSON sent by the client

```
{
  "title": "Fix projector",
  "description": "Meeting room projector is blank.",
  "category": "equipment"
}
```

The form can display “Equipment” and send the agreed value “equipment”.

© 2026 Teerasej Jiraphatchandej. All rights reserved.

Valid JSON can still be rejected

Both examples are well formed. The API checks the values separately.

Accepted category

```
{
  "title": "Fix projector",
  "description": "Meeting room projector is blank.",
  "category": "equipment"
}
```

Unsupported category · 422

```
{
  "title": "Fix projector",
  "description": "Meeting room projector is blank.",
  "category": "urgent"
}
```

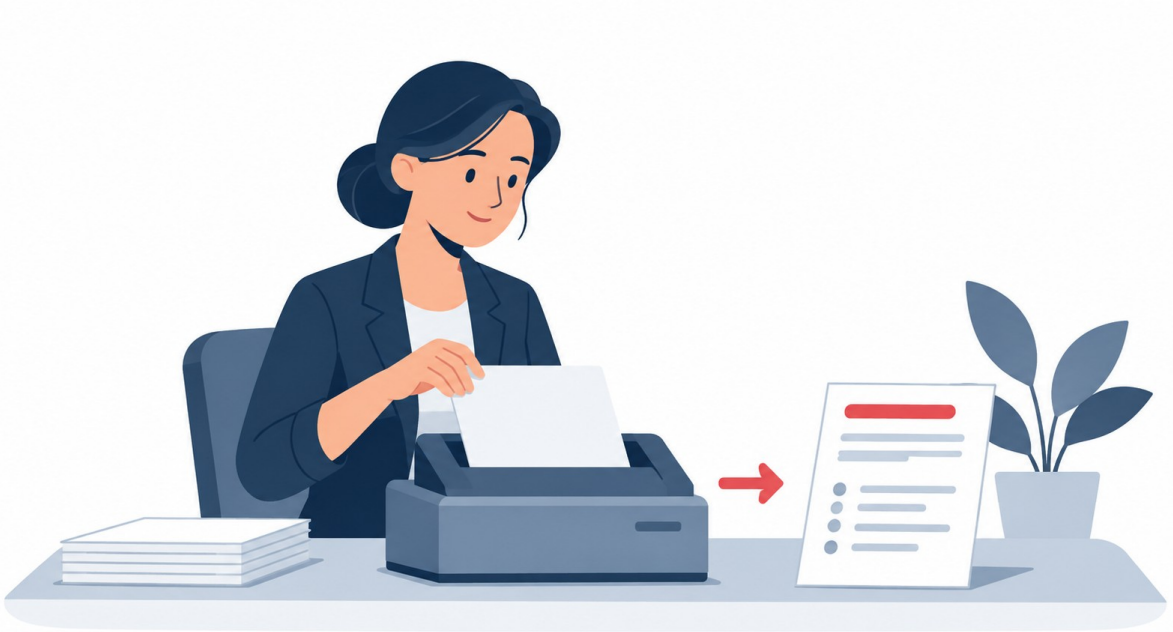
“urgent” is a JSON string, but it falls outside the allowed categories.

© 2026 Teerasej Jiraphatchandej. All rights reserved.

Function

A function handles one defined task

Inputs and outputs make behavior easier to follow



© 2026 Teerasej Jiraphatchandej. All rights reserved.

Class and object

A class defines data and behavior

An object is one instance of that definition



© 2026 Teerasej Jiraphatchandej. All rights reserved.

Module

A module groups related code

Separate files help separate responsibilities



© 2026 Teerasej Jiraphatchandej. All rights reserved.

Virtual environment

A virtual environment isolates project packages

The interpreter must use the intended environment



© 2026 Teerasej Jiraphatchandej. All rights reserved.

FastAPI and the server

FastAPI defines routes and behavior

A running server process receives requests



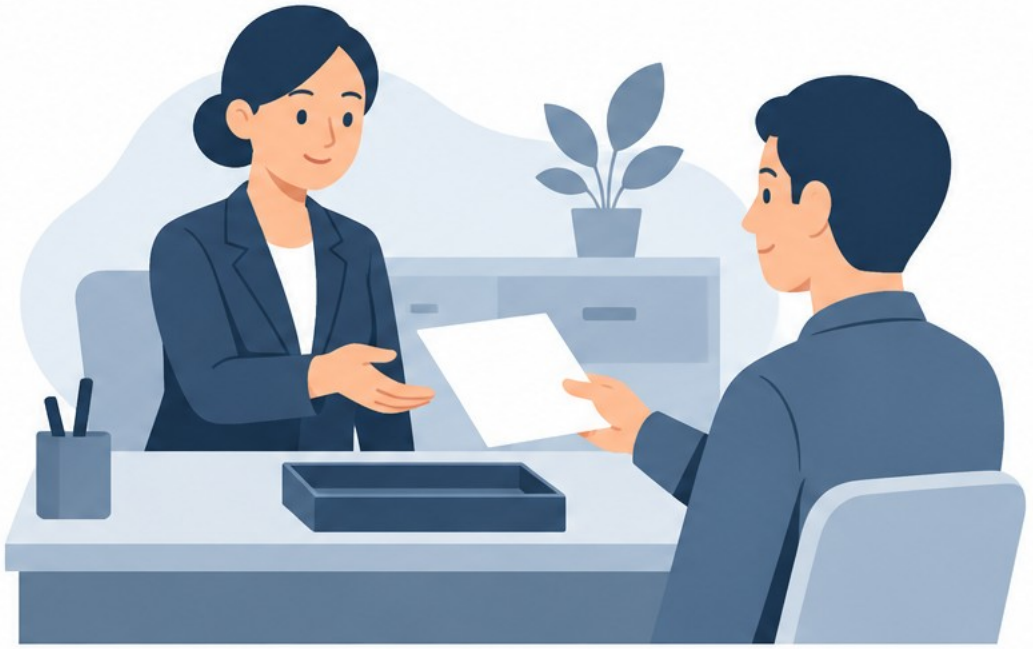
© 2026 Teerasej Jiraphatchandej. All rights reserved.

A handbook and an open counter



FastAPI

Defines the route and reply.



Uvicorn

Runs the app and receives requests.

© 2026 Teerasej Jiraphatchandej. All rights reserved.

What a health check tells you

A browser calls the running API and gets a small reply.

Browser

GET /health

→

Uvicorn

Receives the call

→

FastAPI

Returns HTTP 200

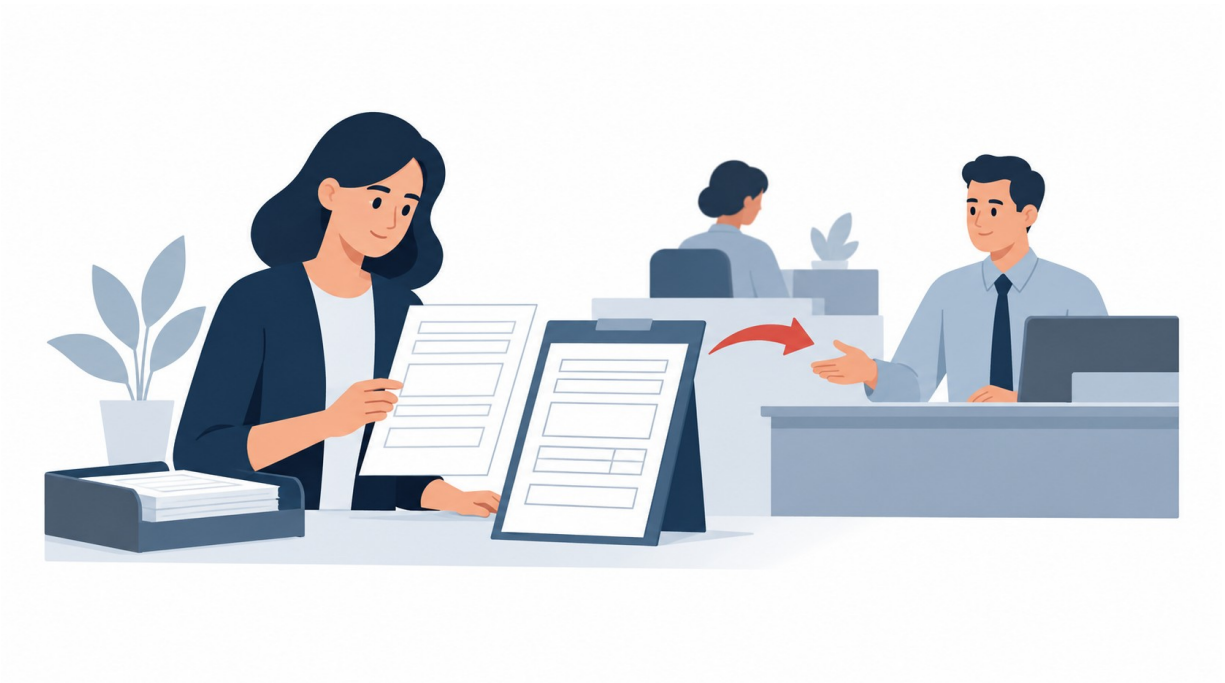
200 OK { "status": "ok", "version": "v1" }

A database check comes later, when you save and read a request.

© 2026 Teerasej Jiraphatchandej. All rights reserved.

Exercise 2

Routes, models and validation

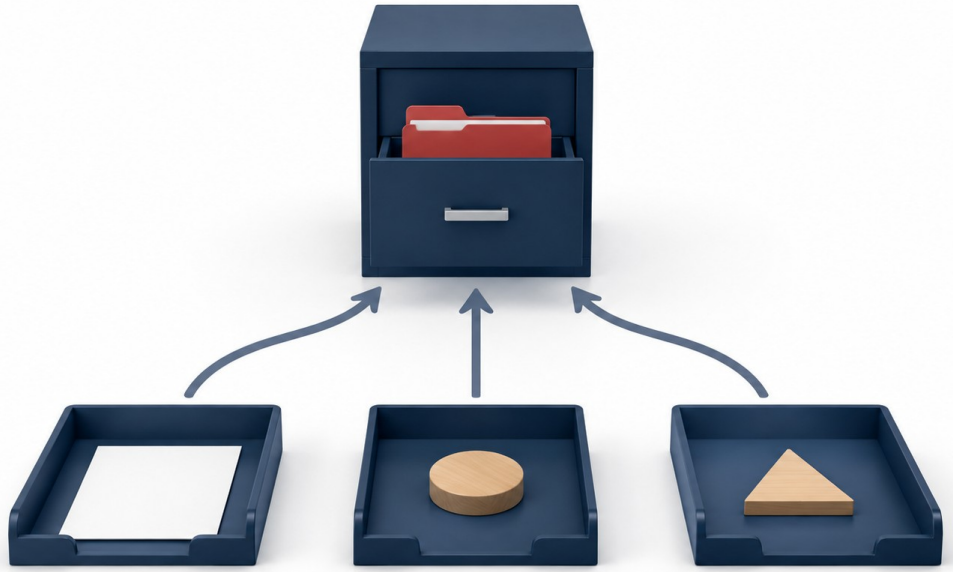


© 2026 Teerasej Jiraphatchandej. All rights reserved.

Route and HTTP method

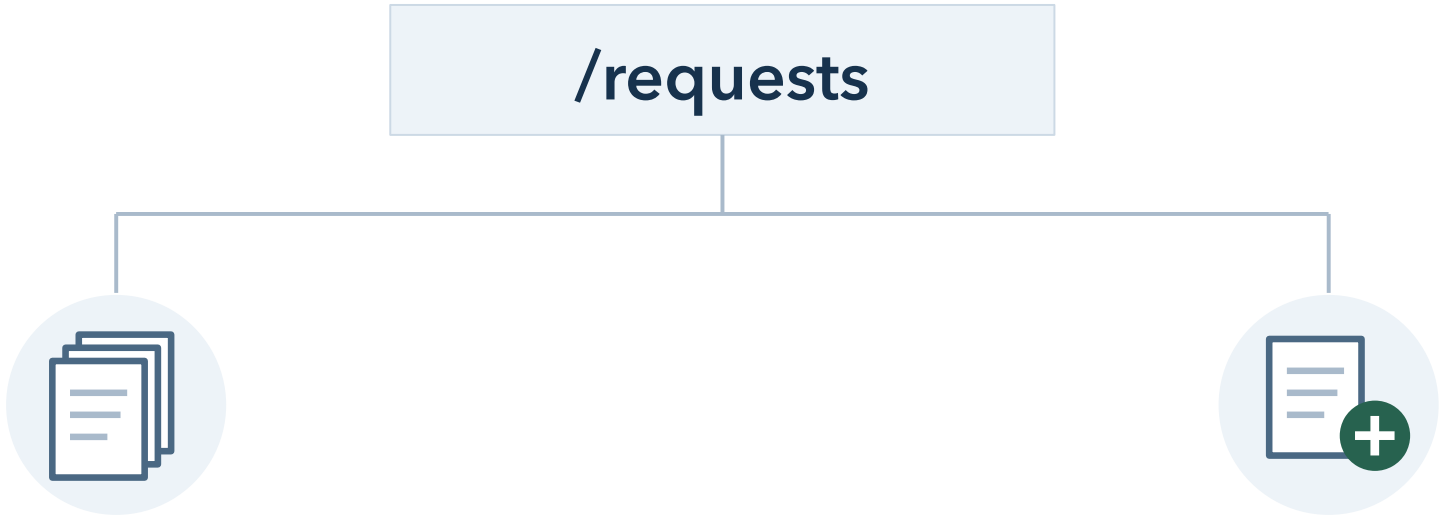
The path identifies the resource

The method describes the action



© 2026 Teerasej Jiraphatchandej. All rights reserved.

One path, two actions



GET

List requests → 200 OK

POST

Create a request → 201 Created

© 2026 Teerasej Jiraphatchandej. All rights reserved.

Read or update one request



GET /requests/{id}

Read the request.

PATCH /requests/{id}/status

{ "status": "done" }

© 2026 Teerasej Jiraphatchandej. All rights reserved.

Request model

The client sends only accepted fields

CreateRequest accepts title, description and category



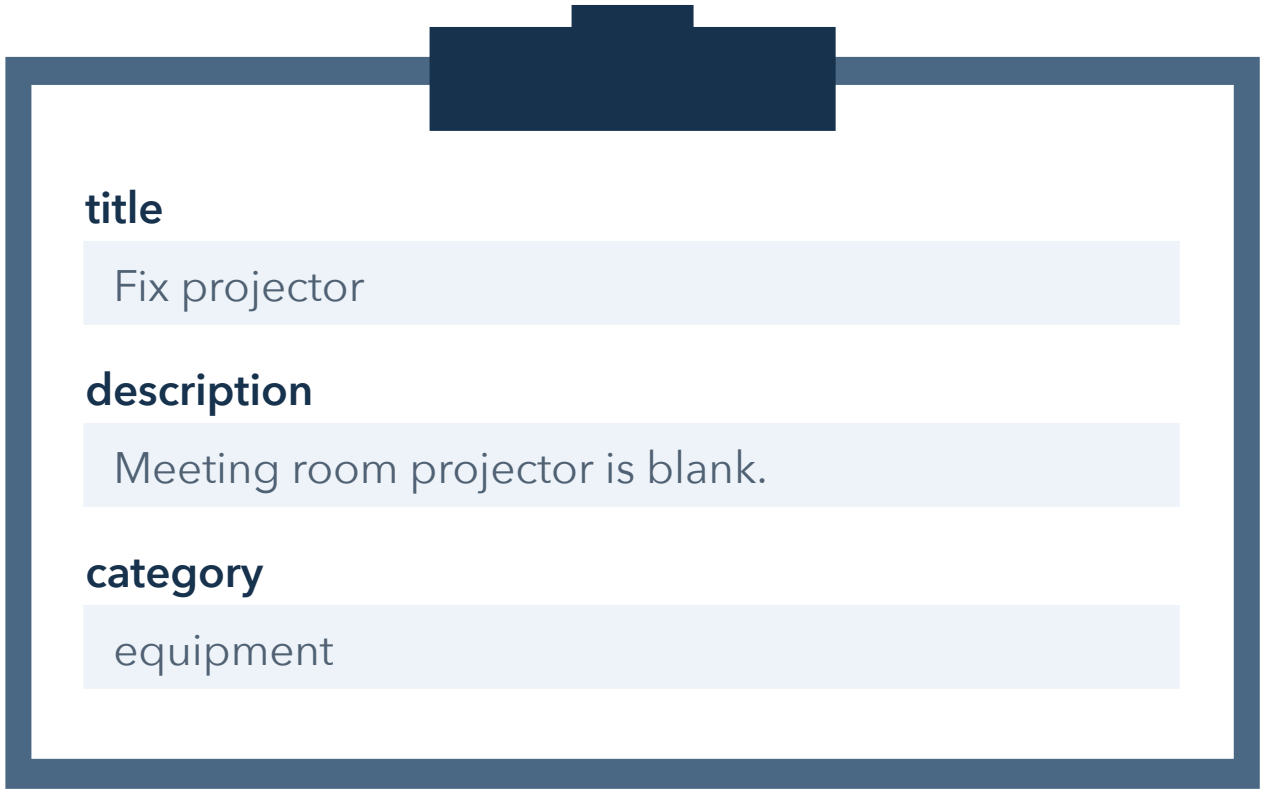
© 2026 Teerasej Jiraphatchandej. All rights reserved.

What the client puts on the form

CreateRequest

The client fills three fields.





© 2026 Teerasej Jiraphatchandej. All rights reserved.

Who assigns the owner?



Client adds an owner

owner_username: learner02

422 · Extra field



API assigns the owner

Verified token → learner01

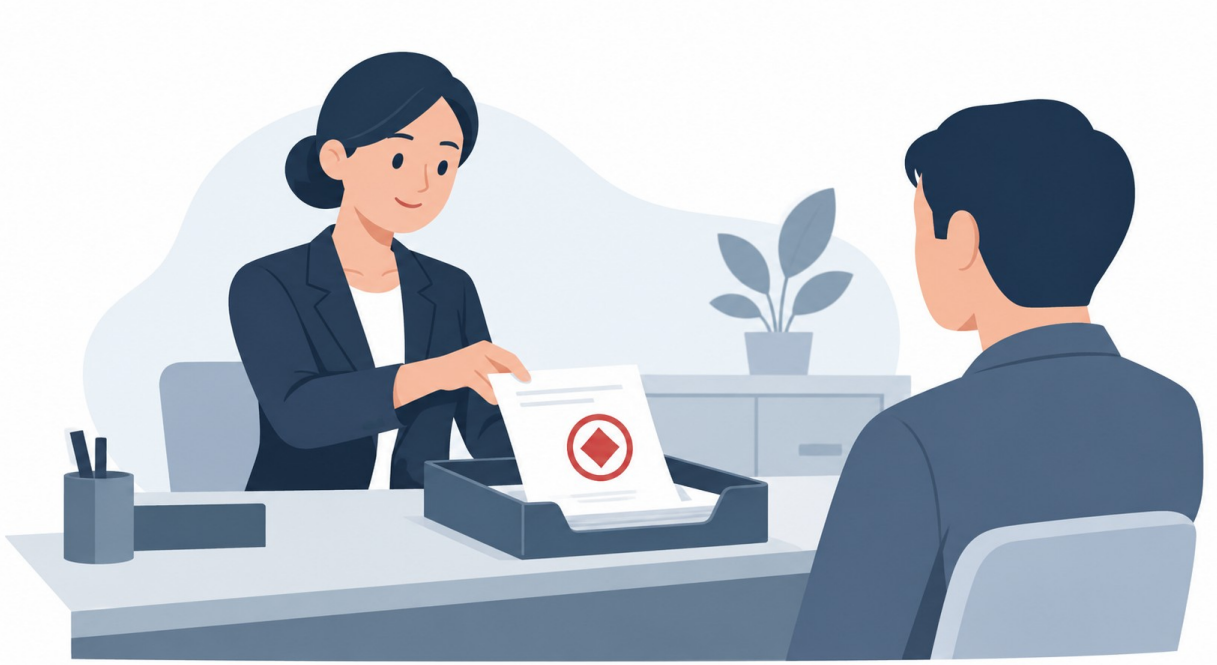
After login · owner_username: learner01

© 2026 Teerasej Jiraphatchandej. All rights reserved.

Response model

The response reports confirmed data

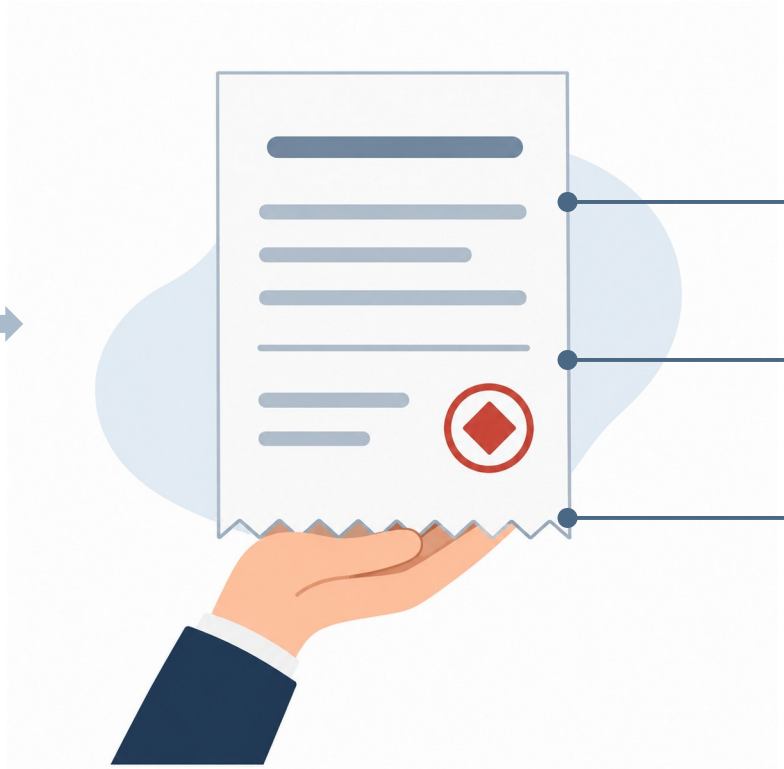
A UUID links later actions to the same request



© 2026 Teerasej Jiraphatchandej. All rights reserved.

The response is your receipt

201 Created



Request form

id

UUID of the saved request

status: new

Initial status

created_at

UTC timestamp

© 2026 Teerasej Jiraphatchandej. All rights reserved.

One receipt, one UUID



Create

Receive the ID

Read

status: new

Update

status: done

Same UUID · 7b7cd82c-378e-4ccb-9f2b-c983bd2ba7e1

© 2026 Teerasej Jiraphatchandej. All rights reserved.

Server-owned fields

The server assigns id and created_at

A verified token determines owner_username



© 2026 Teerasej Jiraphatchandej. All rights reserved.

Validation

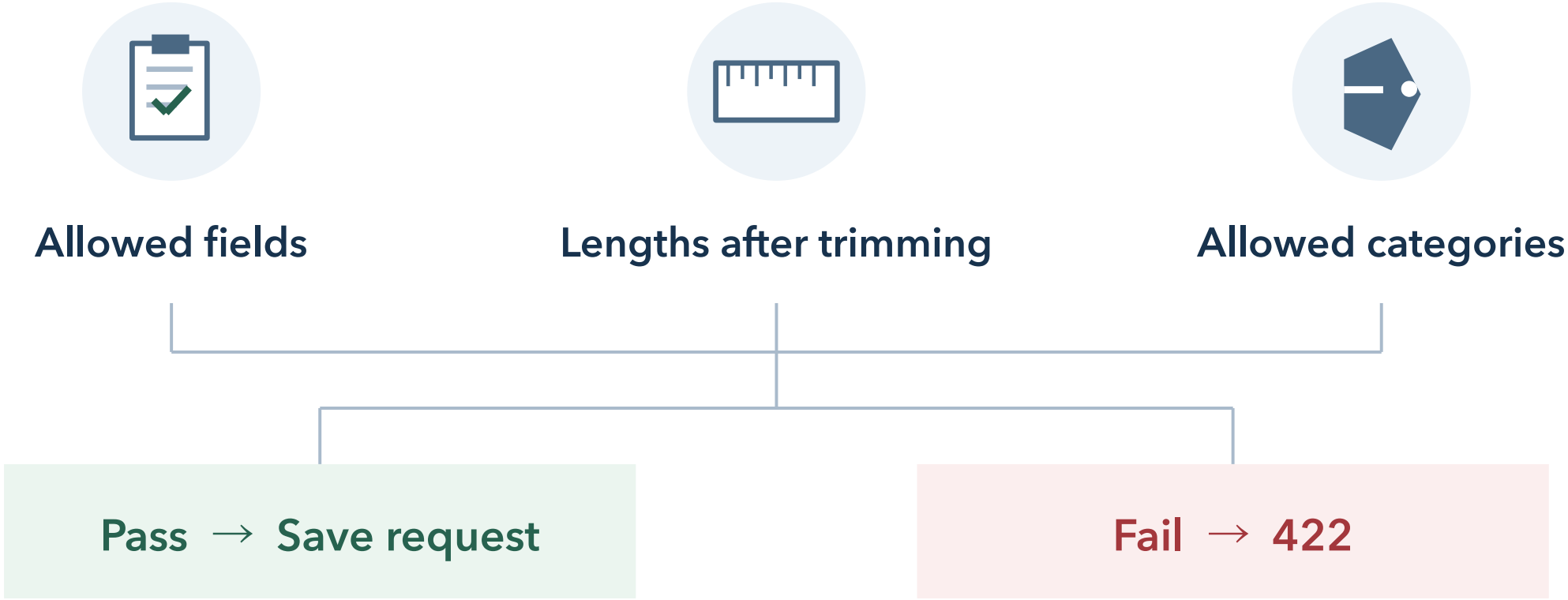
Input must meet the rules before use

Length, allowed values and fields form the contract



© 2026 Teerasej Jiraphatchandej. All rights reserved.

Validation checks the form before saving



© 2026 Teerasej Jiraphatchandej. All rights reserved.

A 422 points to the field to fix

201 · Created

title

Fix projector

422 · Fix the title

title

" " (three spaces)

Error location ["body", "title"]

© 2026 Teerasej Jiraphatchandej. All rights reserved.

HTTP status codes

Status codes describe the result



This lab uses 201 for creation and 422 for invalid input

© 2026 Teerasej Jiraphatchandej. All rights reserved.

OpenAPI

OpenAPI describes the API's structure

A shared schema keeps clients and services aligned

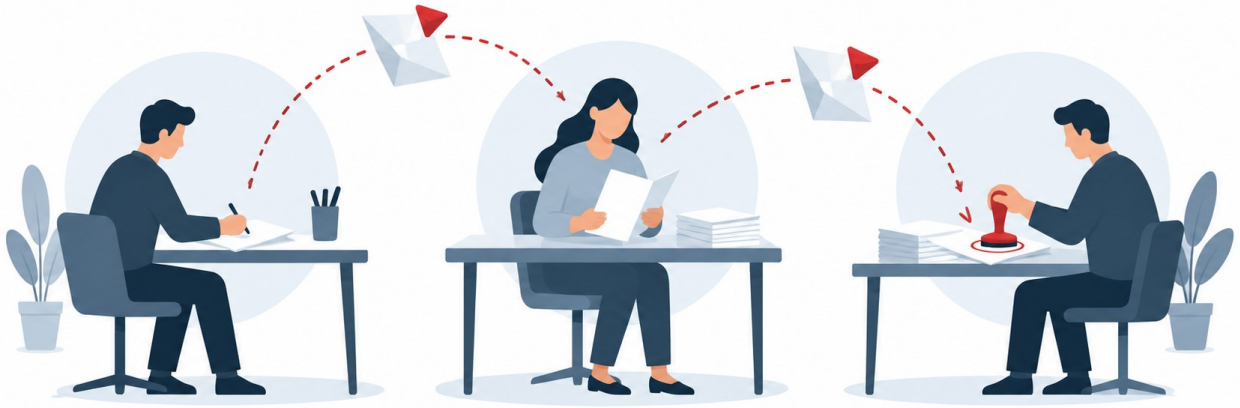


© 2026 Teerasej Jiraphatchandej. All rights reserved.

One request through the API

Create, read and update share one ID

A successful response does not prove persistence



© 2026 Teerasej Jiraphatchandej. All rights reserved.

Exercise 3

Store data in PostgreSQL

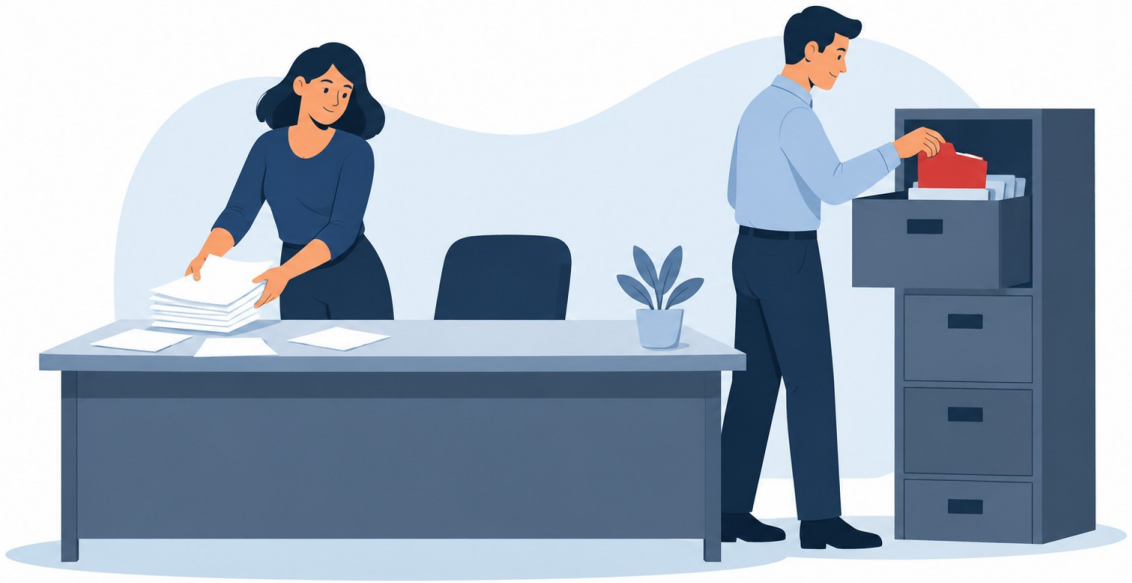


© 2026 Teerasej Jiraphatchandej. All rights reserved.

Memory and persistence

Memory belongs to a running process

PostgreSQL stores data for later use



© 2026 Teerasej Jiraphatchandej. All rights reserved.

Schema

A schema defines the data structure

Types and relationships keep records consistent

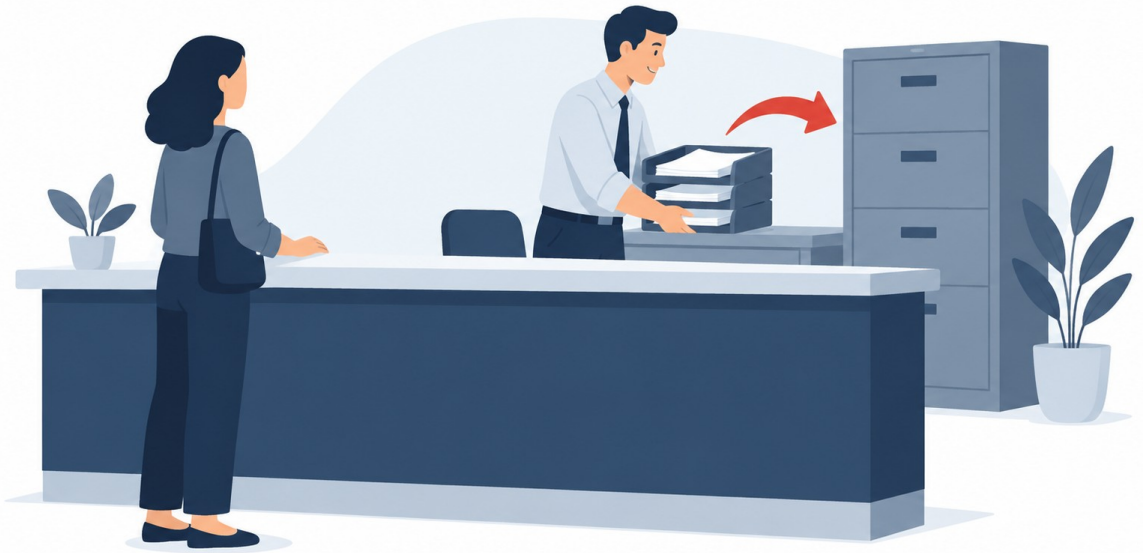


© 2026 Teerasej Jiraphatchandej. All rights reserved.

Repository layer

The repository layer handles data access

Routes can stay stable when storage changes



© 2026 Teerasej Jiraphatchandej. All rights reserved.

Parameter binding

SQL instructions and input values stay separate

The database driver binds values to the query



© 2026 Teerasej Jiraphatchandej. All rights reserved.

Transaction

Related writes succeed or fail together

Commit confirms the work; rollback cancels it



© 2026 Teerasej Jiraphatchandej. All rights reserved.

Configuration and network

Database settings belong on the API side

Host, TLS and network access must also be correct

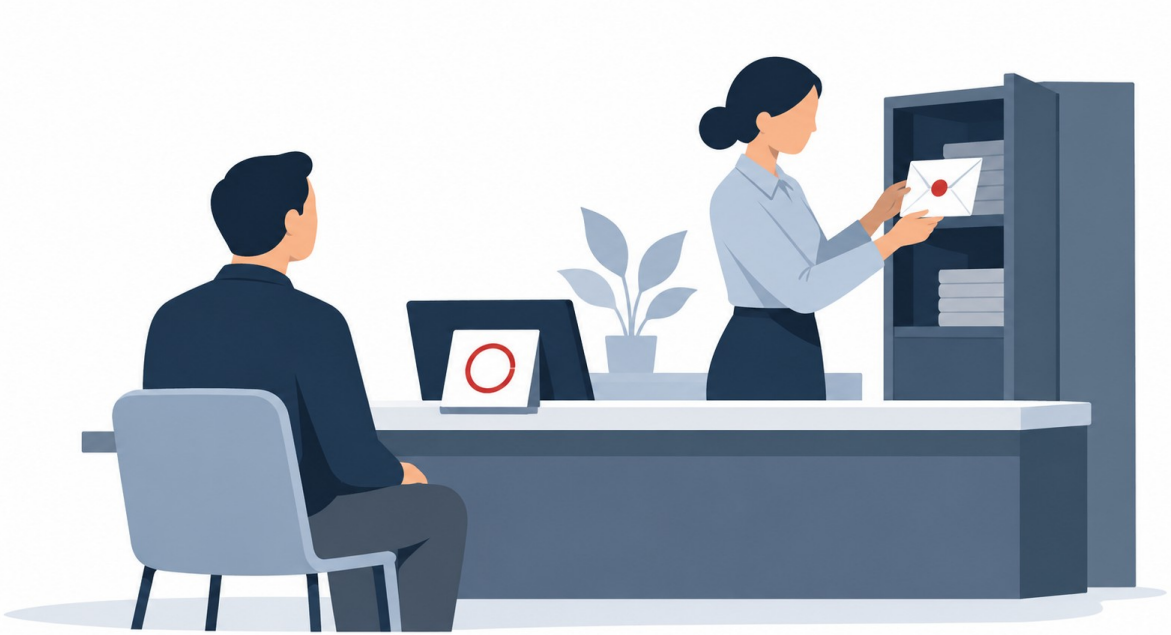


© 2026 Teerasej Jiraphatchandej. All rights reserved.

Evidence of persistence

The same UUID survives a restart

Process health and database access prove different things



© 2026 Teerasej Jiraphatchandej. All rights reserved.

Exercise 4

Login, ownership and tests



© 2026 Teerasej Jiraphatchandej. All rights reserved.

Authentication and ownership

Authentication checks who the user is

Ownership limits which requests they can access



© 2026 Teerasej Jiraphatchandej. All rights reserved.

Two checks before opening the record



Verify identity

Who is signed in?



Check ownership

Does the record belong to them?

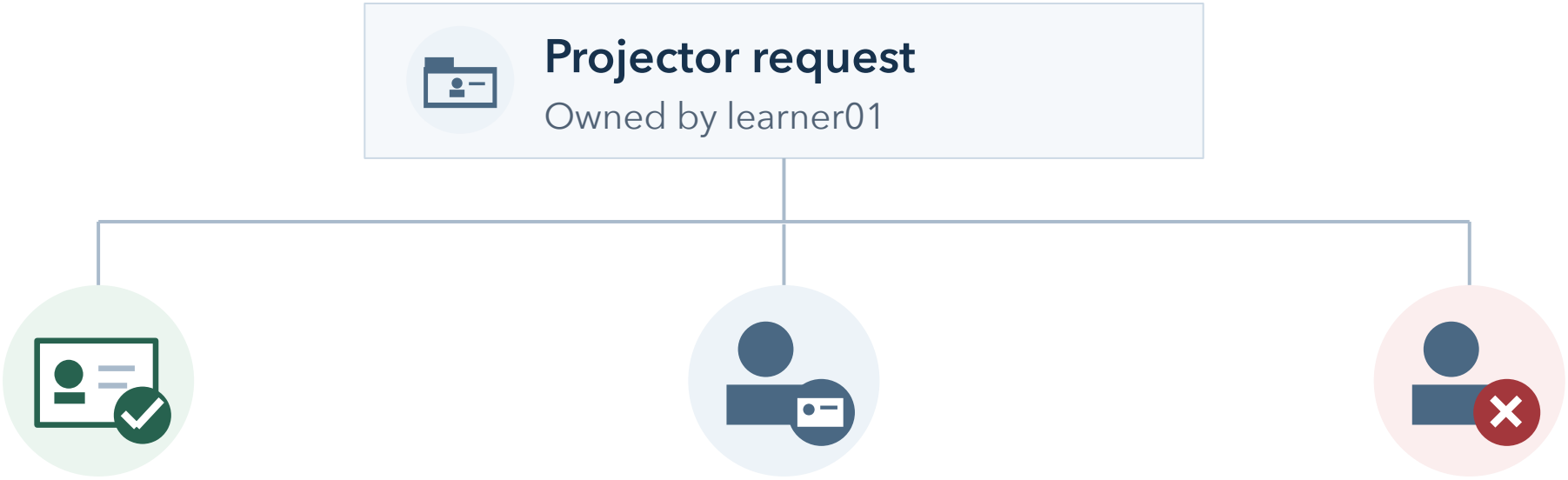


Return the record

Both checks must pass.

© 2026 Teerasej Jiraphatchandej. All rights reserved.

Three visitors, one request

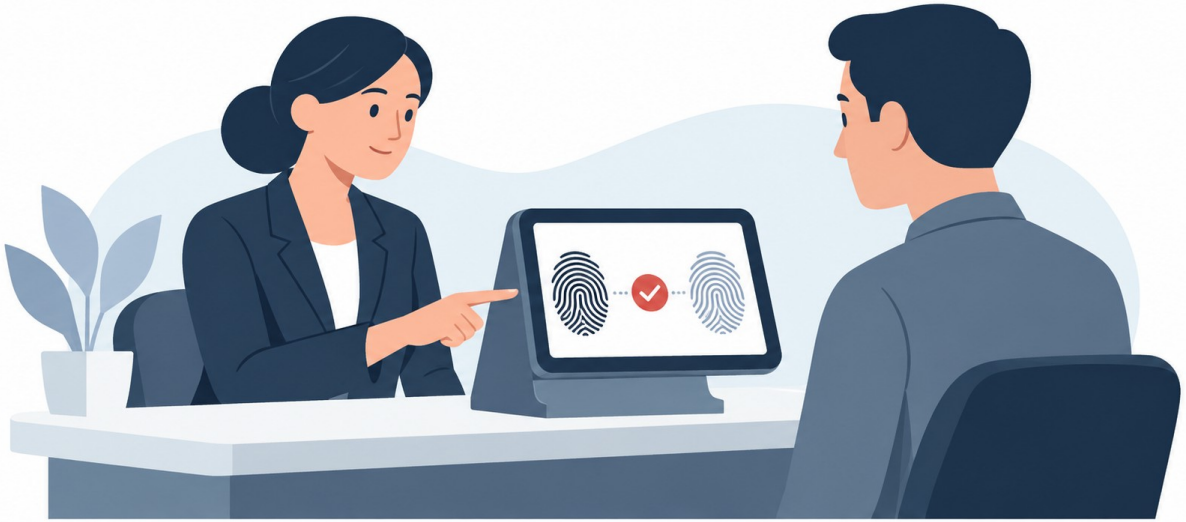


© 2026 Teerasej Jiraphatchandej. All rights reserved.

Password hashing

Store a hash instead of a plain password

A hash supports verification, not decryption



© 2026 Teerasej Jiraphatchandej. All rights reserved.

Bearer token and JWT

A token avoids sending the password each time

The API checks its signature and expiry



© 2026 Teerasej Jiraphatchandej. All rights reserved.

Sign in once, show the pass later

At login

On later requests







Sign in

POST /auth/token

Receive a pass

API returns an access token

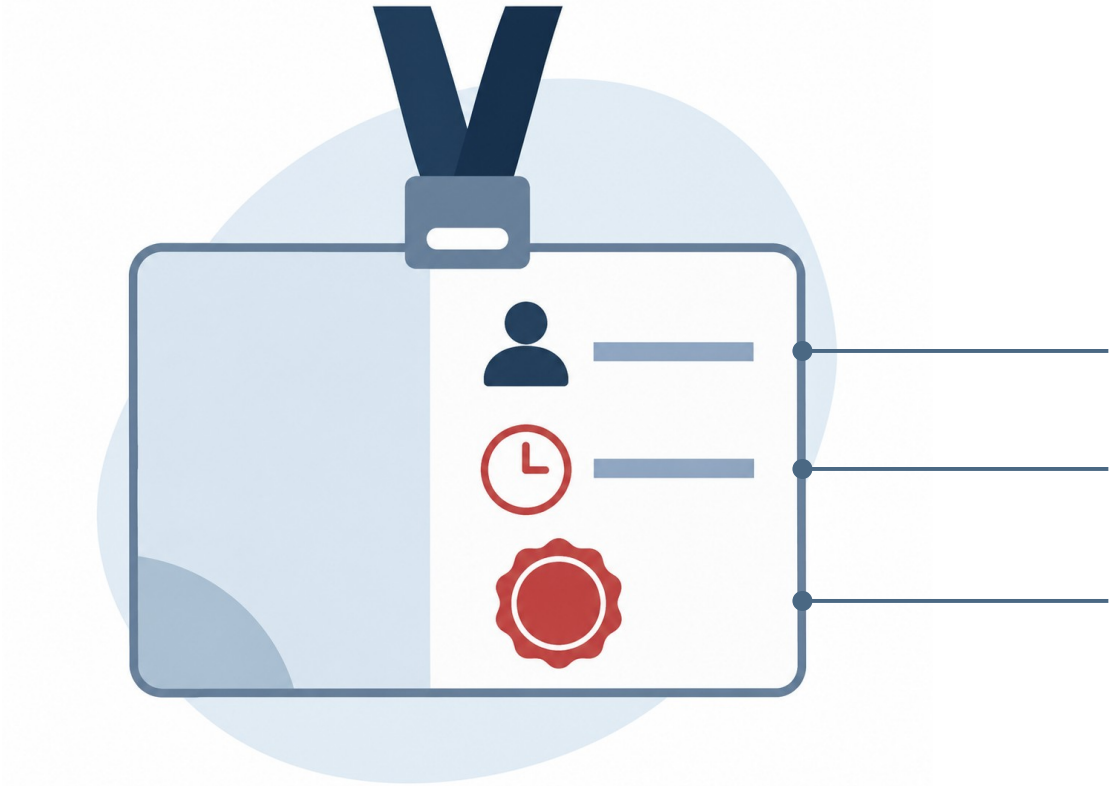
Show the pass

GET /requests

Authorization: Bearer <token>

© 2026 Teerasej Jiraphatchandej. All rights reserved.

What makes a temporary pass valid?



sub · Name
Identifies the signed-in user.

exp · Deadline
Sets when the token expires.

Signature · Stamp
Lets the API verify the signed content.

Expired token → 401

© 2026 Teerasej Jiraphatchandej. All rights reserved.

Errors without data leaks

Different failures need different responses

Useful error details must not expose secrets



© 2026 Teerasej Jiraphatchandej. All rights reserved.

Automated tests

Tests repeat checks of expected behavior

Rejected requests matter as much as successful ones



© 2026 Teerasej Jiraphatchandej. All rights reserved.

What an API test actually checks



Arrange

Valid projector payload



Act

POST /requests



Assert

Check the code and body

HTTP 201

status: new

© 2026 Teerasej Jiraphatchandej. All rights reserved.

Test the allowed and rejected paths



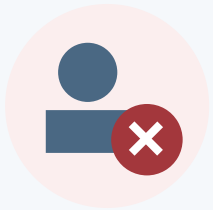
Valid creation

201 · Created



Invalid input

422 · Validation error



Missing bearer token

401 · Authentication required



Another learner's request

404 · Request not found

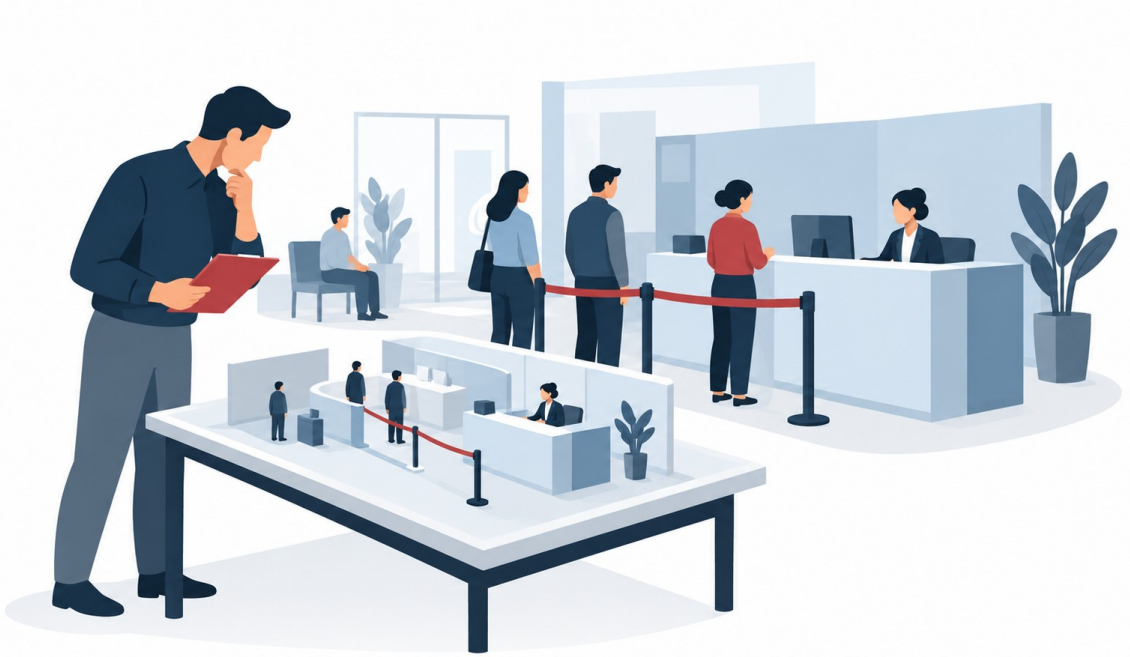
Run all four cases with the same offline test setup.

© 2026 Teerasej Jiraphatchandej. All rights reserved.

Different kinds of evidence

Unit tests isolate behavior with test doubles

Live checks exercise the API and database together

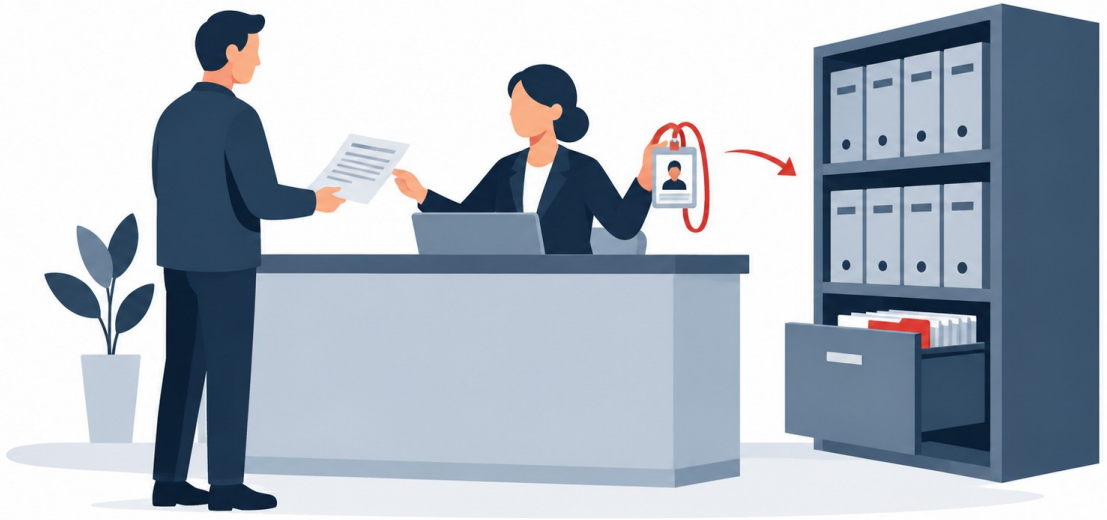


© 2026 Teerasej Jiraphatchandej. All rights reserved.

An API the app can rely on

Contract, storage and access rules work together

Evidence shows which parts have been verified



© 2026 Teerasej Jiraphatchandej. All rights reserved.